



cpi'fp

Los Enlaces

# DESARROLLO WEB EN ENTORNO SERVIDOR

2º DESARROLLO DE APLICACIONES WEB

PRIMEROS PASOS LARAVEL

## 1. ARTISAN

**Consola de comandos Artisan:** Automatiza muchas tareas habituales al trabajar con Laravel.

- Generar esqueletos de controladores y modelos.
- Crear migraciones de bases de datos.
- Rellenar BD con datos de prueba.
- Hacer el enrutamiento.
- ...

## 1. ARTISAN

### Un ejemplo práctico: crear un controlador.

- **A mano:** vamos al directorio `/app/Http/Controllers` y creamos un archivo llamado, digamos, *HolaController.php*. Luego lo rellenamos con el esqueleto de un controlador vacío, copiando y pegando de otro controlador existente y eliminando todo lo que no nos haga falta.

- **Con Artisan:**

Artisan creará el archivo `/app/Http/Controllers/HolaController.php` y lo rellenará con el esqueleto de un controlador vacío y sintácticamente correcto.

## 1. ARTISAN

### Comandos principales de Artisan:

- **php artisan list:** listado de todos los comandos válidos en tu instalación de Laravel.
- **php artisan db:migrate:** para hacer migraciones (crear la estructura de nuestra base de datos).
- **php artisan db:seed:** para llenar de datos predefinidos nuestra base de datos.

## 1. ARTISAN

### Comandos principales de Artisan:

- **php artisan make:migration** → crea una migración (para crear la estructura de la base de datos).
- **php artisan make:seeder** → crea un seeder (para rellenar con datos las tablas).
- **php artisan make:controller** → para crear un controlador.
- **php artisan make:model** → para crear un modelo.
- **php artisan route:list** → muestra todas las rutas definidas.

## 2. PRIMEROS PASOS

### Crear una clave segura:

- Lo primero que hay que hacer con cualquier aplicación nueva es ejecutar este comando, **Laravel no funcionará sin ella.**
- Esto crea una clave de encriptación aleatoria que se guarda en el archivo de configuración de la aplicación (.env). Laravel utilizará esa clave para cifrar ciertas cosas (por ejemplo, las contraseñas de usuario).

## 2. PRIMEROS PASOS

### Probar el enrutador:

- Para capturar una ruta podemos editar el archivo `/routes/web.php` y añadir las siguientes líneas de código:

```
Route::get('/hola', function() {  
    return "Hola, mundo.";  
});
```

Cuando escribas la ruta “hola” en la barra de direcciones del navegador, se ejecutará esta función anónima (también denominadas *closure*) y, como resultado, se verá el texto “Hola, mundo.” en la ventana del navegador. Pruébalo en el navegador: <http://localhost/laravel/example-app/public/index.php/hola>

## 2. PRIMEROS PASOS

### Crear una ruta hacia un controlador:

- Los *closures* o funciones sin nombre raramente se usan en el enrutador. Lo que suele hacer el enrutador es redirigir la ejecución hacia un controlador. Es justo lo que vamos a hacer ahora. Edita el enrutador (`/routes/web.php`) y sustituye la ruta anterior por esto:

```
Route::get('/hola', 'HolaController@index');
```



## 2. PRIMEROS PASOS

### Crear una ruta hacia un controlador:

- Esto indica al enrutador que, al recibir la ruta “hola”, se debe ejecutar el método *index()* del controlador *HolaController*. Pero el controlador *HolaController* no existe, así que vamos a crearlo con este comando de Artisan:

```
Route::get('/hola', 'HolaController@index');
```

## 2. PRIMEROS PASOS

### Crear una ruta hacia un controlador:

- Editamos el controlador (`/app/Http/Controllers/HolaController.php`) y le añadimos el método `index()`. Comprueba que funciona en el navegador: <http://localhost/laravel/example-app/public/index.php/hola>

```
public function index() {  
    return "Hola, mundo";  
}
```

## 2. PRIMEROS PASOS

### Crear una ruta hacia un controlador:

- A partir de Laravel 8, para que este enrutamiento funcione es necesario modificar el archivo `/app/Providers/RouteServiceProvider.php`:

```
$this->routes(function () {  
    Route::prefix('api')  
        ->middleware('api')  
        ->namespace('App\Http\Controllers') <----- AÑADE ESTO  
        ->group(base_path('routes/api.php'));  
    ...  
});
```

## 2. PRIMEROS PASOS

### Crear una ruta hacia un controlador:

- Y también (/app/Providers/RouteServiceProvider.php):

```
$this->routes(function () {  
    ...  
    Route::middleware('web')  
        ->namespace($this->namespace)  
        ->namespace('App\Http\Controllers') <----- AÑADE ESTO  
        ->group(base_path('routes/web.php'));  
});
```

## 2. PRIMEROS PASOS

### Crear una ruta hacia un controlador:

- Actualmente, el enrutamiento se hace con:

```
// Enrutador
Route::get('/hola', HolaController::class);

// Controlador
public function __invoke() {
    return "Hola, mundo";
}
```

## 2. PRIMEROS PASOS

### Cargar una vista desde el controlador:

- En la arquitectura MVC, el controlador no debería producir ninguna salida HTML. Vamos a modificarlo para que el método *index()* del controlador no genere el “Hola, mundo”, sino que invoque a una vista que se encargue de ello.
- Además, vamos a inyectar en la URL una variable con el nombre del usuario para mostrar cómo se capturan esos valores y cómo se pasan a las vistas en Laravel.

## 2. PRIMEROS PASOS

### Cargar una vista desde el controlador:

- Comenzamos modificando el enrutador (*/routes/web.php*). Observa cómo se usan las llaves, { y }, para indicar la presencia de una variable en la URL:

```
Route::get('/hola/{nombre}', 'HolaController@show');
```

## 2. PRIMEROS PASOS

### Cargar una vista desde el controlador:

- Ahora creamos un método `show()` en el controlador (`/app/Http/Controllers/HolaController.php`). Como ves, estamos invocando una vista llamada `hola` y le estamos pasando un array con los datos necesarios (el nombre del usuario, en este caso).

```
public function show($nombre) {  
    $data['nombre'] = $nombre;  
    return view('hola', $data);  
}
```



## 2. PRIMEROS PASOS

### Cargar una vista desde el controlador:

- Esa vista debe crearse en `/resources/views/hola.blade.php` y puede tener este aspecto:

```
<body>
    Saludos, {{ $nombre }}.
    Ha ingresado en el sitio web DWES.
</body>
```

Puedes probar esta nueva ruta cargando en el navegador una ruta como:

<http://localhost/laravel/example-app/public/index.php/hola/ProfesorFalken>

## 3. ENRUTADOR

El **enrutador** de Laravel es el componente que captura las URL solicitadas al servidor y las traduce a **invocaciones de métodos de los controladores**.

El enrutador es capaz, además, de **mapear fragmentos de la URL** a variables PHP que serán inyectadas como parámetros a los métodos del controlador.

## 3. ENRUTADOR

Esto significa que, si le pides al servidor una ruta como ésta:

`https://mi-servidor/user/delete/12`

...el enrutador puede “trocear” esa URL para extraer los segmentos (“user”, “delete” y “12”). Y tú puedes decidir qué hacer con cada uno de esos segmentos. Lo normal en este ejemplo sería que invocaras el método *delete()* del controlador *UserController*, y que ese método recibiera como parámetro el dato 12, que será el id del usuario que se pretende borrar.

## 3. ENRUTADOR

### Enrutamiento básico:

Como hemos visto en el ejemplo “Hola, mundo”, hay varias formas de generar una salida HTML desde el enrutador (*/routes/web.php*). En este código de ejemplo vemos las cuatro formas básicas:

```
// Forma 1: generar la salida directamente en el enrutador, con un closure  
(función sin nombre)  
Route::get('/hola', function() {  
    return "Hola, mundo";  
});
```

## 3. ENRUTADOR

### Enrutamiento básico:

// Forma 2: llamar a una función de un controlador sin pasarle parámetros

```
Route::get('/hola', 'HolaController@show');
```

// Forma 3: llamar a una función de un controlador pasándole parámetros

```
Route::get('/hola/{nombre}', 'HolaController@show');
```

// Forma 4: llamar a una función de un controlador con un parámetro optativo

```
Route::get('/hola/{nombre?}', 'HolaController@show');
```

## 3. ENRUTADOR

La diferencia entre la forma 3 y la 4 es que, en la forma 3, la ruta debe llevar forzosamente un dato a continuación de “/hola” (algo como: `https://mi-servidor/hola/juan`). Si no lo lleva, el enrutador considerará que no se trata de esa ruta y seguirá buscando alguna ruta coincidente en el resto del archivo.

En cambio, en la forma 4, el dato final es optativo, así que el enrutador invocará el método `show()` del controlador tanto si ese dato aparece en la URL como si no lo hace.

## 3. ENRUTADOR

### Rutas con nombre:

Es MUY recomendable asignar un nombre a las rutas en el enrutador. Esto hace que, más adelante, podamos cambiar la URL de los enlaces sin tener que modificar el código fuente de nuestras vistas.

El nombre se le asigna añadiendo `->name('nombre')` al final.

```
Route::get('/contactar', 'ContactController@contact')->name('contact')
```

## 3. ENRUTADOR

En tu código fuente, debes referirte a esta ruta siempre con la expresión `route('contact')` (ya veremos exactamente cómo se hace esto), pero el usuario verá la dirección `https://servidor/contactar`.

En el futuro se puede cambiar la forma en la que lo ve el usuario. Por ejemplo, puedes cambiar `Route::get('/contactar'...)` por `Route::get('/acerca-de'...)`, pero no tendrás que modificar ni una línea de código más en tu aplicación, porque internamente esa ruta seguirá llamándose `route('contact')`.



## 3. ENRUTADOR

### Verbos http:

Además de GET, en el enrutador se pueden enrutar otras acciones...

```
Route::get();      // Solicitudes habituales
Route::post();     // Recepción de datos de formulario (para INSERT)
Route::put();      // Recepción de datos para UPDATE (también puede escribirse
Route::patch(),   que no es lo mismo, pero casi)
Route::delete();  // Recepción de datos para DELETE
Route::patch(array('GET','POST'), 'ruta', acción) // Responderá tanto a GET
como a POST
```

## 3. ENRUTADOR

**GET:** se utiliza para obtener recursos del servidor. Por ejemplo, si queremos obtener información de un usuario, podemos utilizar una ruta el verbo GET.

**POST:** se utiliza para crear recursos en el servidor. Por ejemplo, si queremos crear un nuevo usuario en la base de datos, podemos utilizar una ruta con el verbo POST.

**PUT:** se utiliza para actualizar recursos completos en el servidor. Por ejemplo, si queremos actualizar toda la información de un usuario en la base de datos, podemos utilizar una ruta con el verbo PUT.

## 3. ENRUTADOR

**PATCH:** se utiliza para actualizar parte de un recurso en el servidor. Por ejemplo, si queremos actualizar solamente el correo electrónico de un usuario en la base de datos, podemos utilizar una ruta con el verbo PATCH.

**DELETE:** se utiliza para eliminar recursos del servidor. Por ejemplo, si queremos eliminar un usuario de la base de datos, podemos utilizar una ruta con el verbo DELETE.

Los verbos **PUT**, **PATCH** y **DELETE** no están soportados aún por HTML.

## 3. ENRUTADOR

No se puede crear un formulario así: `<form method='PUT'>`, porque el navegador no lo entenderá. Solo puedes poner `<form method='GET'>` o `<form method='POST'>`.

Cuando trabajes con Laravel, puedes emular PUT, PATCH o DELETE en los formularios así:

```
<form action="/foo/bar" method="POST">  
    @method('DELETE')
```

<https://laracoding.com/sending-a-form-to-a-put-route-in-laravel-with-example/>

## 3. ENRUTADOR

### Orden de las rutas:

El orden en el que se escriben las rutas en el enrutador es importante.

Por ejemplo, si pedimos la dirección *http://mi-servidor/usuario/crear*, escribir estas dos rutas en este orden es un error:

```
Route::get('usuario/{nombre}', 'UserController@show');  
Route::get('usuario/crear', 'UserController@create');  
//El enrutador tratará de mostrar un usuario cuyo nombre sea "crear" (que seguramente no  
existirá) porque la petición encaja con las dos rutas y el enrutador elegirá la primera  
ruta que encuentre.
```

## 3. ENRUTADOR

La solución pasa por alterar el orden de las líneas en el enrutador.

De este modo, la petición *http://mi-servidor/usuario/crear* seguirá encajando en las dos rutas, pero el enrutador elegirá la primera.

En cambio, una petición parecida con cualquier otro nombre (por ejemplo, *http://mi-servidor/usuario/luis*), solo encajará con la segunda ruta.

```
Route::get('usuario/crear', 'UsuarioController@create');  
Route::get('usuario/{nombre}', 'UsuarioController@show');
```

## 4. SERVIDOR RESTFUL

Un servidor RESTful es aquel que responde a la [arquitectura REST](#).

La arquitectura **REST** no es más que una forma estandarizada de construir un servidor para que realice las tareas típicas de mantenimiento de recursos. Y los recursos pueden ser cualquier cosa que se almacene en el servidor: usuarios, clientes, productos, películas, facturas...

Es decir: el 99% de las veces, los recursos son registros en una tabla de la base de datos.

## 4. SERVIDOR RESTFUL

El enrutador de un servidor RESTful contendrá las 7 operaciones definidas en la arquitectura REST para cada recurso accesible desde la red, y que permiten manipular el recurso:

- Mostrarlo (un único recurso o todos)
- Buscarlo (un único recurso o todos)
- Insertarlo
- Modificarlo
- Borrarlo



## 4. SERVIDOR RESTFUL

Por ejemplo, para un recurso llamado “user”, esas 7 operaciones son:

```
Route::get('user', 'UserController@index')->name('user.index');           // Recupera todos los usuarios
Route::get('user/{user}', 'UserController@show')->name('user.show');       // Recupera usuario con id=user
Route::get('user/crear', 'UserController@create')->name('user.create');     // Lanza el formulario de creación de
usuarios
Route::post('user/{user}', 'UserController@store')->name('user.store');     // Recoge los datos del
formulario y los inserta en la base de datos
Route::get('user/{user}/edit', 'UserController@edit')->name('user.edit');   // Lanza el formulario de
modificación de usuarios
Route::patch('user/{user}', 'UserController@update')->name('user.update');  // Recoge los datos del
formulario y modifica el usuario de la base de datos
Route::delete('user/{user}', 'UserController@destroy')->name('user.destroy'); // Elimina al usuario de la
base de datos
```

## 4. SERVIDOR RESTFUL

Ten en cuenta que, si estás construyendo un servidor RESTful, debes respetar escrupulosamente los nombres y URLs de las rutas. Así, cualquier otro usuario o aplicación que use tu servidor sabrá cómo manipular los recursos sin necesidad de consultar la documentación.

Laravel te permite resumir esas entradas del enrutador en una sola línea que engloba a las siete rutas REST:

```
Route::resource('user');
```

## 5. CONTROLADORES

Los **controladores** son un elemento clave de la **arquitectura MVC**. En Laravel, los controladores funcionan exactamente igual que en las aplicaciones MVC hechas con PHP clásico. Es decir, son los puntos de entrada a la aplicación desde el enrutador.

Los controladores deberían permanecer lo más sencillos posible: nada de accesos a la base de datos ni de generación de HTML. Esas acciones se derivarán a los modelos y las vistas. El controlador es un organizador del flujo de la aplicación: decide qué componente tiene que trabajar y el orden en el que lo debe hacer.

## 5. CONTROLADORES

1. Los controladores en Laravel siempre heredan de la clase *Controller* o de una subclase de *Controller*.
2. Su nombre debería escribirse en singular, CamelCase y terminando en la palabra Controller: *UserController*, *LoginController*...
3. Cada método del controlador debe terminar en un *return*. Lo que el método devuelva será convertido automáticamente en una *HTTP response 200*, es decir, en una respuesta válida http, excepto si es un array, en cuyo caso Laravel lo convertirá automáticamente en JSON.

## 5. CONTROLADORES

Los controladores se pueden crear a mano: vas al directorio `/app/http/controllers`, creas allí un archivo vacío y empiezas a escribir código. Pero nadie lo hace así porque Artisan ya crea el esqueleto del archivo por ti. Así que es mejor que vayas a lo práctico: abre una consola en tu servidor web y ponte a escribir.

### Forma 1. Crear un controlador vacío.

```
$ php artisan make:controller UserController
```

## 5. CONTROLADORES

### Forma 2. Crear un controlador de tipo resource.

Estos controladores se generan automáticamente con un andamiaje para construir recursos **REST**. Es decir, la clase ya llevará incorporados los métodos *index()*, *create()*, *store()*, *show()*, *edit()*, *update()* y *destroy()* del estándar REST.

Para crear un controlador RESTful:

```
$ php artisan make:controller UserController --resource
```

## 5. CONTROLADORES

No te olvides de añadir al enrutador (*/routes/web.php*) las rutas REST para este tipo controlador. Te recuerdo que se pueden resumir las siete rutas en esta sola entrada del enrutador:

```
Route::resource('nombreRecurso', 'controlador');
```

En nuestro ejemplo:

```
Route::resource('usuarios', 'UserController');
```

## 5. CONTROLADORES

### Forma 3. Crear un controlador tipo API.

Una **API** (*Application Programming Interface*) es un interfaz entre programas. Es decir, es la forma en la que unos interaccionan con otros.

Algunas aplicaciones web se diseñan para que otros programas las utilicen, no para que las utilicen seres humanos. En estos casos, la interfaz de usuario no existe (o es mínima) y lo importante es la API. Y los métodos del controlador no devuelven vistas, sino datos formateados en JSON.



## 5. CONTROLADORES

Se puede construir un controlador tipo API de forma muy simple, porque es parecido a un *resource* pero sin *create()* ni *edit()*, porque una API no necesita mostrar los formularios de inserción/modificación.

```
$ php artisan make:controller UserController --api
```

De nuevo, no te olvides de las entradas en el enrutador. Puedes englobarlas todas en una sola entrada con este aspecto:

```
Route::apiResource('usuarios', 'UserController');
```